Exercices - Récursivité

Rappels - compléments sur les chaines de caractères en python

Déclaration

Rappel: une chaîne de caractère python peut être initialisée avec une déclaration du type:

```
ma_chaine1 = 'caracteres'
```

Si la chaîne contient un apostrophe, il vaut mieux utiliser une paire de doubles quotes:

```
ma_chaine2 = "l'apostrophe n'est pas interprété comme délimiteur de chaine ici"
```

Enfin, si la chaîne occupe plusieurs lignes, on utilisera une paire de triple quotes:

```
ma_chaine3 = """
ligne 1
ligne 2
etc.
"""
```

```
ma_chaine1 = 'caracteres'
type(ma_chaine1)
```

```
str
```

Indexation et longueur

On peut accéder aux différents caractères de la chaîne en utilisant une indexation comme pour les tableaux. Une particularité de python est de permettre une indexation négative. Ainsi l'index -1 correspond au dernier caractère.

```
ma_chaine2 = "l'apostrophe n'est pas interprété comme délimiteur de chaine ici"
print(ma_chaine2[0])
print(ma_chaine2[-1])
```

```
l
i
```

On obtient la longueur de la chaîne avec la fonction `len`.

```
print(len(ma_chaine2))
```

```
64
```

Les chaines: objets non mutables

Les chaînes de caractères sont des objets non mutables (on dit aussi *immuables*).

```
ma_chaine2[0] = 'L'
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

<ipython-input-50-f4b20dca7b9a> in <module>
----> 1 ma_chaine2[0] = 'L'


TypeError: 'str' object does not support item assignment
```

Tranches (non exigible en NSI)

On peut sélectionner des *tranches* dans une chaîne. Par exemple, pour récupérer les caractères situés entre les index i (*inclus*) et j (**exclus**) d'une chaîne `ch`, on utilise la syntaxe: `ch[i:j]`.

```
ma_chaine2[0:12]#récupère les caractères entre les index 0 (inclus) et 12 (exclus)
```

```
"l'apostrophe"
```

Concaténation

On peut utiliser le signe '+' pour concaténer (mettre bout à bout) deux chaînes. Le résultat est une nouvelle chaîne.

```
new_chaine = 'Les ' + ma_chaine1
print(new_chaine)
```

```
Les caracteres
```

Exercice 1 - Fonction factorielle

La traditionnelle fonction factorielle (pratiquement systématiquement présentée lorsqu'on aborde la récursivité) est définie par: $fact(n) = n! = 1 \times 2 \times \cdots \times n$. Par ailleurs, $0! = 1$.

A faire

Coder une fonction factorielle `fact(n)` de **manière récursive**. Coup de pouce: revoir le codage de la fonction puissance.

```
def fact(n):
    # Supprimer l'instruction 'pass' et compléter avec votre code
    pass
```

Exercice 2 - Inversion de chaîne

On souhaite écrire une fonction `inversechaine` qui inverse tous les caractères d'une chaîne non vide `c`. Par exemple, l'appel `inversechaine('bonjour')` renvoie `'ruojnob'`. Plusieurs méthodes de programmation sont possibles. **On impose ici une méthode récursive**.

- Le cas de base est simple à identifier: il s'agit d'une chaîne de longueur 1 ! Lorsque ce cas se produit, il suffit de renvoyer la chaîne;
- Pour les autres cas, on s'appuie sur le constat suivant: l'inverse d'une chaîne c'est tout simplement la concaténation du dernier caractère de celle-ci avec l'inverse du reste de la chaîne.

A faire

Compléter la fonction suivante qui inverse `c` de manière récursive. Puis tester la fonction en exécutant la cellule prévue à cet effet.

```
def inversechaine(c, fin):
    # Supprimer l'instruction 'pass' et compléter avec votre code
    pass
```

```
# Tests
assert inversechaine('a', 0) == 'a'
assert inversechaine('moteur') == 'ruetom'
```

Exercice 3 - Conversion binaire (encore)

On a vu en première qu'on pouvait obtenir la conversion en binaire d'un entier naturel en réalisant les divisions successives de son quotient par 2. Exprimons cette idée de manière à faire émerger une méthode récursive.
On peut voir la **décomposition en binaire** d'un nombre N comme la **concaténation** de la **décomposition en binaire** du quotient de N par 2, avec son reste. Le cas de base étant obtenu lorsque le nombre N est nul, dans ce cas on renvoie une chaîne vide.

A faire

Compléter la fonction `binaire(N)` qui prend en paramètre un entier naturel N et qui renvoie une chaîne qui correspond à sa conversion en binaire. Une méthode récursive est exigée.

```
def binaire(N):
    # Supprimer l'instruction 'pass' et compléter avec votre code
    pass
```

```
# Test
assert binaire(40) == '101000'
```

Exercice 4 - La recherche linéaire en version récursive

L'algorithme de recherche séquentielle ou linéaire dans un tableau t a été vue en classe de 1re. Donner une implémentation récursive de cet algorithme.

Indication

$x \in t$ si x correspond au caractère en cours ou appartient au reste du tableau.

```
def appartient(x, t, d, f):
    pass
```

```
# Tests
t4 = [1, 2, 10, -5, 8, 6, -2]
t2 = []
assert appartient(-2, t2, 0, len(t2)-1) == False
assert appartient(10, t4, 0, len(t4)-1) == True
```

Exercice 5 (optionnel) - Le tri par insertion

Le tri par insertion a également été vu en 1re. On se propose ici d'implémenter une version récursive.

Amener un élément à sa place dans un tableau trié

Soit un tableau t de n éléments, trié. C'est quoi *amener un élément x à sa place dans t*?

- si t est vide, c'est tout simplement un tableau composé du seul élément x (`[x]`);
- si $x < t[0]$ (plus petit que le premier élément du tableau) alors c'est le tableau obtenu en ajoutant x au début de t;
- si $x \geq t[0]$ c'est la **concaténation** `[t[0]]` avec le tableau résultant du **placement** de x dans `[t[1]...t[n-1]]`.

A faire

Ecrire une fonction **récursive** `place(x,t)` qui amène x à sa place dans un tableau trié. Cette fonction renvoie un nouveau tableau.

```
def place(x, t):
    pass
```

```
# Tests
t=[-5, -3,4,12,15,20]
assert place(8, t) == [-5, -3, 4, 8, 12, 15, 20]
```

Trier par insertion

C'est quoi trier un tableau t de n éléments par insertion ?

- si t est vide alors il s'agit d'un tableau vide;
- sinon c'est **placer** `t[0]` dans le tableau de $n-1$ éléments `[t[1]...t[n-1]]` **trié par insertion**.

A faire

Ecrire la fonction récursive `triInsert(t)` qui trie t par insertion.

```
def triInsert(t):
    pass
```

```python
# Tests
t4 = [1, 2, 10, -5, 8, 6, -2]
assert triInsert(t4) == [-5, -2, 1, 2, 6, 8, 10]
assert triInsert([]) == []
assert triInsert(triInsert([0,0,0])) == [0,0,0]
```

Exercice 6 - Un autre tri: le tri fusion

Préambule

Avant d'aborder l'implémentation d'un nouvel algorithme de tri plus performant que ceux de 1re, on donne les deux fonctions ci-dessous. Malheureusement les spécifications ne sont pas complètes ...

```python
def est_trie(t, g, d):
    """
    t: tableau
    g,d: entiers naturels
    renvoie ...
    """
    # par ordre croissant
    return all(t[i+1] >= t[i] for i in range(g, d))
```

```python
def fusion(t, g, m, d):
    """
    t: tableau
    g,m,d: entiers naturels
    renvoie ....
    """
    assert est_trie(t, g, m)
    assert est_trie(t, m+1, d)

    i, j = g, m + 1
    aux = t[:] # réalise une copie de t
    for k in range(g, d+1):
        if i > m:
            t[k] = aux[j]
            j += 1
        elif j > d:
            t[k] = aux[i]
            i += 1
        elif aux[j] < aux[i]:
            t[k] = aux[j]
            j += 1
        else:
            t[k] = aux[i]
            i += 1

    assert est_trie(t, g, d)
```

```python
# 1er exemple d'appel
t1 = [1, 3, 7, -3, 4, 10, 12]
fusion(t1, 0, 2, 6)
print(t1)
```

```
[-3, 1, 3, 4, 7, 10, 12]
```

```
        k
        |
        v
      +---+---+---+---+---+---+--+
 t    | 1 | 3 | 7 |-3 | 4 |10 |12|
      +---+---+---+---+---+---+--+
        g       m              d

      +---+---+---+---+---+---+--+
 aux  | 1 | 3 | 7 |-3 | 4 |10 |12|
      +---+---+---+---+---+---+--+
        ^           ^
        |           |
        i           j
```

```python
# Autre exemple
t2 = ['E','E','G','M','R','A','C','E','R','T']
fusion(t2,0,4,9)
print(t2)
```

```
['A', 'C', 'E', 'E', 'E', 'G', 'M', 'R', 'R', 'T']
```

A faire

- A partir du code et des exemples, dire ce que réalise la fonction `fusion`.
- Quelle est l'utilité de la fonction `est_trie`?

Le tri fusion

Le tri fusion d'un tableau est basé sur une idée simple:

- séparer le tableau en deux parties égales ou presqu'égales (à un élément près);
- trier de manière indépendante et **récursivement** les deux parties;
- fusionner les deux parties de tableau triées.

Le cas de base se produit lorsqu'on arrive à un tableau à un élément. Dans ce cas, il est forcément trié et on retourne sans plus de traitement.

A faire

En utilisant la cellule suivante, coder une fonction `tri_fusion` et tester.

```
# A compléter

#def tri_fusion(t, g, d):
#    if ...: # Cas de base
#        return
#    else: # cas récursif
#        m = g + (d - g) // 2 # sensiblement la moitié du tableau
#        ...
```

```
# Cellule de test
t1 = [3, -8, 10,1, 3, -5, 12, 7, 6]
t2 = []
tri_fusion(t1, 0, 8)
tri_fusion(t2, 0, 0)
assert t1 == [-8, -5, 1, 3, 3, 6, 7, 10, 12]
assert t2 == []
```

Problème classique: les tours de Hanoï

Origine: wikipedia

Le problème mathématique des tours de Hanoï a été inventé par Édouard Lucas. Il est publié dans le tome 3 de ses Récréations mathématiques, parues à titre posthume en 1892. Il annonce que ce problème est dû à un de ses amis, N. Claus de Siam (anagramme de Lucas d'Amiens, Amiens étant sa ville de naissance), prétendument professeur au collège de Li-Sou-Stian (anagramme de Saint Louis, le lycée où Lucas enseignait).

Sous le titre « Les brahmes tombent », Lucas relate que « N. Claus de Siam a vu, dans ses voyages pour la publication des écrits de l'illustre Fer-Fer-Tam-Tam, dans le grand temple de Bénarès, au-dessous du dôme qui marque le centre du monde, trois aiguilles de diamant, plantées dans une dalle d'airain, hautes d'une coudée et grosses comme le corps d'une abeille. Sur une de ces aiguilles, Dieu enfila au commencement des siècles, 64 disques d'or pur, le plus large reposant sur l'airain, et les autres, de plus en plus étroits, superposés jusqu'au sommet. C'est la tour sacrée du Brahmâ. Nuit et jour, les prêtres se succèdent sur les marches de l'autel, occupés à transporter la tour de la première aiguille sur la troisième, sans s'écarter des règles fixes que nous venons d'indiquer, et qui ont été imposées par Brahma. Quand tout sera fini, la tour et les brahmes tomberont, et ce sera la fin des mondes1 ! ».

Comme indiqué ci-dessous, un jeu à 64 disques requiert un minimum de $2^{64} - 1$ déplacements. En admettant qu'il faille 1 seconde pour déplacer un disque, ce qui fait 86 400 déplacements par jour, la fin du jeu aurait lieu au bout d'environ 213 000 milliards de jours, ce qui équivaut à peu près à 584,5 milliards d'années, soit 43 fois l'âge estimé de l'univers (13,7 milliards d'années selon certaines sources).

```python
from IPython.display import YouTubeVideo


YouTubeVideo('U3nGNJTxYc4', width=800, height=450)
```

L'objectif est de résoudre ce casse-tête en utilisant une méthode récursive. Le cas de base est obtenu lorsque le nombre n de disques à déplacer est nul. Dans ce cas on retourne.
Le cas récursif est à faire apparaître avec la remarque suivante: pour déplacer n disques d'une tour A vers une tour C, il faut:

- déplacer les $n-1$ premiers disques vers une tour intermédiaire (B par exemple);
- déplacer le plus grand disque restant vers sa destination (ici tour C);
- déplacer les $n-1$ disques de la première étape vers C.

A faire

Proposer une solution récursive, en python, du problème des tours de Hanoï. La fonction pourrait avoir une signature du type: `hanoi(nb_disques, tour_depart, tour interm, tour_arrivee)`.

```python
def hanoi(n, tour_A, tour_B, tour_C):
    """
    déplace n disques de 'tour_A' vers 'tour_C', en utilisant la tour
    intermédiaire 'tour_B'
    """
    # Supprimer l'instruction 'pass' et compléter avec votre code
    pass
```